

สมาชิกในกลุ่ม

รหัสนักศึกษา	ชื่อ-นามสกุล
673380292-5	ศิขรินทร์ อุปจันทร์
673380301-0	แทนคุณ พันธุ์นิกุล
673380294-1	สรวิชญ์ ศาสนสุพันธ์

ภาพรวมการทดลอง

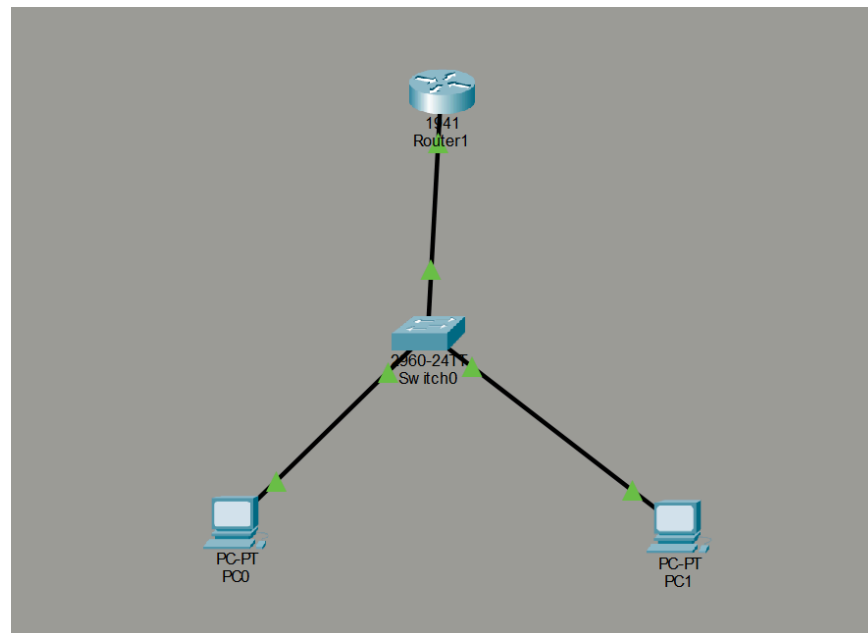
การทดลองนี้เป็นการศึกษาการทำงานของเครือข่ายที่ผสมผสานระหว่างการกำหนดค่าเครือข่ายคลาสสิกกับวิศวกรรมชั้นแอปพลิเคชัน โดยจะได้ทำการกำหนดค่า inter-VLAN routing, ปรับใช้ MIME-aware socket server, จับแพ็กเก็ต และวิเคราะห์การไหลของข้อมูลผ่านชั้นต่างๆ ของโปรโตคอล

สถานการณ์จำลอง: Client ใน VLAN หนึ่งทำการถ่ายโอนไฟล์ไปยัง Server ในอีก VLAN หนึ่งโดยใช้โปรโตคอล MIME-based (JSON header + binary payload) โดยจะได้ตรวจสอบการเชื่อมต่อ, สังเกตการ encapsulation, และวิเคราะห์แพ็กเก็ตโดยใช้ Wireshark

การทดลองนี้ใช้เทคโนโลยี Router-on-a-Stick โดยมีอุปกรณ์ดังนี้:

PC1 (Client) → VLAN 10 → Switch S1 → Trunk → Router R1

PC2 (Server) → VLAN 20 → Switch S1 → Trunk → Router R1



ตารางการกำหนดที่อยู่ IP

อุปกรณ์	Interface	IP Address	Gateway
R1	G0/0.10	192.168.10.1/24	-
R1	G0/0.20	192.168.20.1/24	-
PC1-Client	NIC	192.168.10.10	192.168.10.1
PC2-Server	NIC	192.168.20.20	192.168.20.1

ส่วนที่ 1: การกำหนดค่าอุปกรณ์เครือข่าย

ขั้นตอนที่ 1: การกำหนดค่า Router (R1)

ทำการกำหนดค่า Router-on-a-Stick โดยสร้าง subinterface สำหรับแต่ละ VLAN พร้อมกำหนด encapsulation dot1Q และ IP address:

```
enable
```

```
configure terminal
```

```
hostname R1
```

```
interface g0/0
```

```
no shutdown
```

```
interface g0/0.10
```

```
encapsulation dot1Q 10
```

```
ip address 192.168.10.1 255.255.255.0
```

```
interface g0/0.20
```

```
encapsulation dot1Q 20
```

```
ip address 192.168.20.1 255.255.255.0
```

```
end
```

```
write memory
```

```
Router>enable
Router#configure terminal
Enter configuration commands, one per line. End with CNTL/Z.
Router(config)#hostname R1
R1(config)#interface g0/0
R1(config-if)#no shutdown

R1(config-if)#interface g0/0.10
R1(config-subif)#encapsulation dot1Q 10
R1(config-subif)#ip address 192.168.10.1 255.255.255.0
R1(config-subif)#interface g0/0.20
R1(config-subif)#encapsulation dot1Q 20
R1(config-subif)#ip address 192.168.20.1 255.255.255.0
R1(config-subif)#end
R1#write memory
Building configuration...
[OK]
R1#
%LINK-5-CHANGED: Interface GigabitEthernet0/0, changed state to up
```

ขั้นตอนที่ 2: การกำหนดค่า Switch (S1)

ทำการสร้าง VLAN 10 และ VLAN 20 พร้อมกำหนดค่า trunk port และ access port:

```
enable
```

```
configure terminal
```

```
hostname S1
```

```
vlan 10
```

```
name CLIENTS
```

```
vlan 20
```

```
name SERVERS
```

```
interface g0/1
```

```
switchport mode trunk
```

```
interface fa0/1
```

```
switchport mode access
```

```
switchport access vlan 10
```

```
interface fa0/2
```

```
switchport mode access
```

```
switchport access vlan 20
```

```
end
```

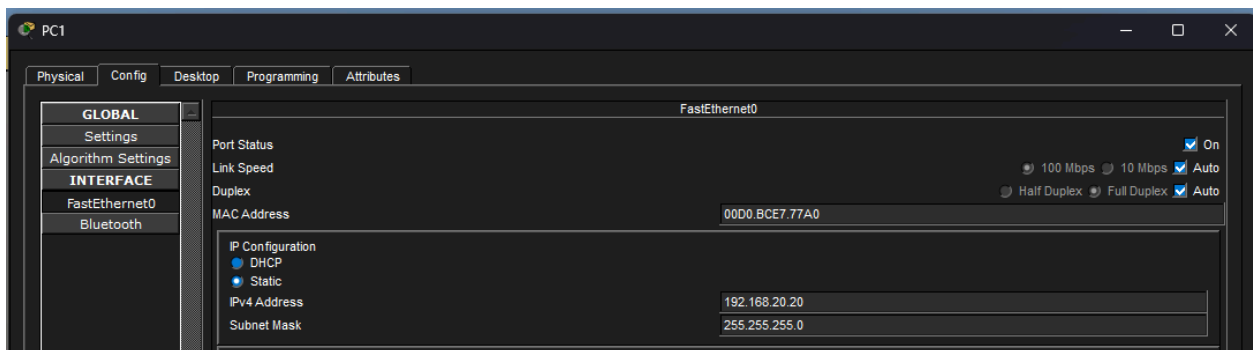
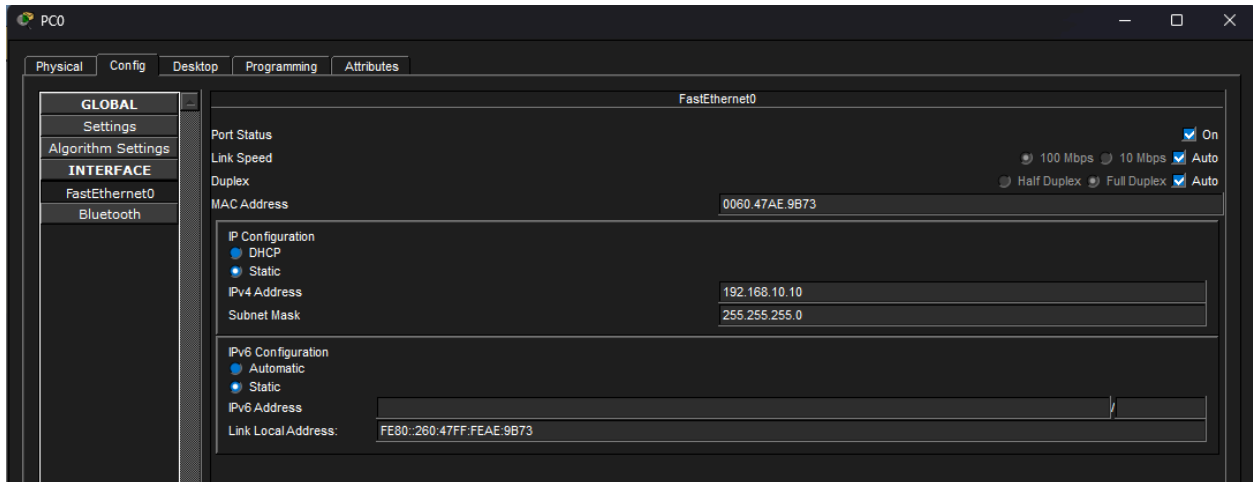
```
write memory
```

```
Switch>enable
Switch#configure terminal
Enter configuration commands, one per line.  End with CNTL/Z.
Switch(config)#hostname S1
S1(config)#vlan 10
S1(config-vlan)#name CLIENTS
S1(config-vlan)#vlan 20
S1(config-vlan)#name SERVERS
S1(config-vlan)#interface g0/1
S1(config-if)#switchport mode trunk

S1(config-if)#interface fa0/1
S1(config-if)#switchport mode access
S1(config-if)#switchport access vlan 10
S1(config-if)#interface fa0/2
S1(config-if)#switchport mode access
S1(config-if)#switchport access vlan 20
S1(config-if)#end
S1#write memory
Building configuration...
[OK]
S1#
%LINEPROTO-5-UPDOWN: Line protocol on Interface GigabitEthernet0/1, changed state to down
```

ขั้นตอนที่ 3: การกำหนดค่า Host

ทำการกำหนดค่า IP Address แบบ Static ตามตารางด้านบน โดย PC1 (Client) ใช้ IP 192.168.10.10/24 และ PC2 (Server) ใช้ IP 192.168.20.20/24



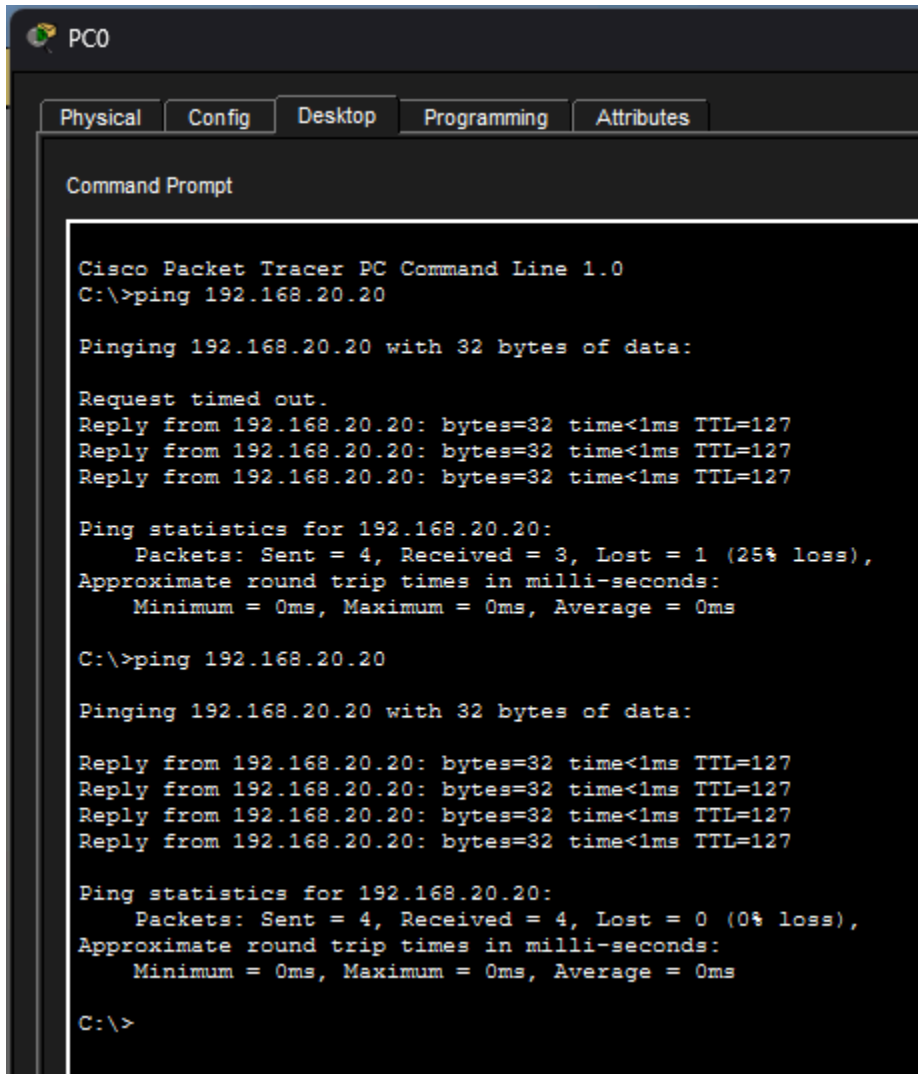
ส่วนที่ 2: ตรวจสอบการเชื่อมต่อ

การทดสอบด้วยคำสั่ง Ping

ทำการทดสอบการเชื่อมต่อจาก PC1 (192.168.10.10) ไปยัง PC2-Server (192.168.20.20) โดยใช้คำสั่ง ping:

```
ping 192.168.20.20
```

ผลการทดลอง: การ ping สำเร็จ โดยได้รับ Reply จาก 192.168.20.20 ทุก packet แสดงว่า inter-VLAN routing ทำงานถูกต้อง



```
PC0
Physical Config Desktop Programming Attributes
Command Prompt

Cisco Packet Tracer PC Command Line 1.0
C:\>ping 192.168.20.20

Pinging 192.168.20.20 with 32 bytes of data:

Request timed out.
Reply from 192.168.20.20: bytes=32 time<1ms TTL=127
Reply from 192.168.20.20: bytes=32 time<1ms TTL=127
Reply from 192.168.20.20: bytes=32 time<1ms TTL=127

Ping statistics for 192.168.20.20:
    Packets: Sent = 4, Received = 3, Lost = 1 (25% loss),
    Approximate round trip times in milli-seconds:
        Minimum = 0ms, Maximum = 0ms, Average = 0ms

C:\>ping 192.168.20.20

Pinging 192.168.20.20 with 32 bytes of data:

Reply from 192.168.20.20: bytes=32 time<1ms TTL=127
Reply from 192.168.20.20: bytes=32 time<1ms TTL=127
Reply from 192.168.20.20: bytes=32 time<1ms TTL=127
Reply from 192.168.20.20: bytes=32 time<1ms TTL=127

Ping statistics for 192.168.20.20:
    Packets: Sent = 4, Received = 4, Lost = 0 (0% loss),
    Approximate round trip times in milli-seconds:
        Minimum = 0ms, Maximum = 0ms, Average = 0ms

C:\>
```

การตรวจสอบบน Router

ใช้คำสั่ง show ip route และ show ip interface brief เพื่อตรวจสอบ routing table และสถานะของ interface:

show ip route

show ip interface brief

ผลการทดลอง: พบเส้นทางไปยัง network 192.168.10.0/24 และ 192.168.20.0/24 โดยตรง (C - connected) และ subinterface G0/0.10 และ G0/0.20 มีสถานะเป็น up/up

```
R1#show ip route
Codes: L - local, C - connected, S - static, R - RIP, M - mobile, B - BGP
       D - EIGRP, EX - EIGRP external, O - OSPF, IA - OSPF inter area
       N1 - OSPF NSSA external type 1, N2 - OSPF NSSA external type 2
       E1 - OSPF external type 1, E2 - OSPF external type 2, E - EGP
       i - IS-IS, L1 - IS-IS level-1, L2 - IS-IS level-2, ia - IS-IS inter area
       * - candidate default, U - per-user static route, o - ODR
       P - periodic downloaded static route

Gateway of last resort is not set

    192.168.10.0/24 is variably subnetted, 2 subnets, 2 masks
C       192.168.10.0/24 is directly connected, GigabitEthernet0/0.10
L       192.168.10.1/32 is directly connected, GigabitEthernet0/0.10
    192.168.20.0/24 is variably subnetted, 2 subnets, 2 masks
C       192.168.20.0/24 is directly connected, GigabitEthernet0/0.20
L       192.168.20.1/32 is directly connected, GigabitEthernet0/0.20

R1#show ip interface brief
Interface                IP-Address      OK? Method Status      Protocol
GigabitEthernet0/0       unassigned      YES unset    up          up
GigabitEthernet0/0.10    192.168.10.1    YES manual   up          up
GigabitEthernet0/0.20    192.168.20.1    YES manual   up          up
GigabitEthernet0/1       unassigned      YES unset    administratively down down
Vlan1                    unassigned      YES unset    administratively down down
R1#
```

การตรวจสอบบน Switch

ใช้คำสั่ง show interfaces trunk และ show mac address-table เพื่อตรวจสอบ trunk configuration และ MAC address table:

show interfaces trunk

show mac address-table

ผลการทดลอง: Port G0/1 ทำงานในโหมด trunk และอนุญาต VLAN 10 และ 20 ส่วน MAC address table แสดง MAC address ของ PC1 และ PC2

พร้อม VLAN ID ที่ถูกต้อง

```
S1#show interfaces trunk
Port      Mode      Encapsulation  Status        Native vlan
Gig0/1    on        802.1q         trunking      1

Port      Vlans allowed on trunk
Gig0/1    1-1005

Port      Vlans allowed and active in management domain
Gig0/1    1,10,20

Port      Vlans in spanning tree forwarding state and not pruned
Gig0/1    1,10,20

S1#show mac address-table
          Mac Address Table
-----
Vlan    Mac Address      Type      Ports
----    -
1       0060.7013.ce01   DYNAMIC   Gig0/1
10      0060.47ae.9b73   DYNAMIC   Fa0/1
10      0060.7013.ce01   DYNAMIC   Gig0/1
20      0060.7013.ce01   DYNAMIC   Gig0/1
20      00d0.bce7.77a0   DYNAMIC   Fa0/2
S1#
```

ส่วนที่ 3: การปรับใช้ MIME Server

การรับ MIME Server บน PC2

ทำการรัน MIME Socket Server บน PC2 (Server) โดยใช้คำสั่ง:

```
cd automation
```

```
python server/main_enhanced.py --host 0.0.0.0 --port 65432 --verbose
```

ผลการทดลอง: Server เริ่มทำงานและรอรับการเชื่อมต่อบน 0.0.0.0:65432 แสดงข้อความ "Server listening on 0.0.0.0:65432"

```
PS C:\GitHub\Clone\week01-mime-typing> python server/main_enhanced.py --host 0.0.0.0 --port 65432 --verbose
[22:49:36] INFO: Starting server with host=0.0.0.0, port=65432, storage=storage
[22:49:36] INFO: Server listening on 0.0.0.0:65432
|
```

การรัน MIME Client บน PC1

ทำการรัน MIME Client บน PC1 (Client) เพื่อส่งไฟล์ sample.png ไปยัง Server:

```
cd automation
```

```
python client/main_enhanced.py assets/sample.png --host 192.168.20.20 --port 65432
```

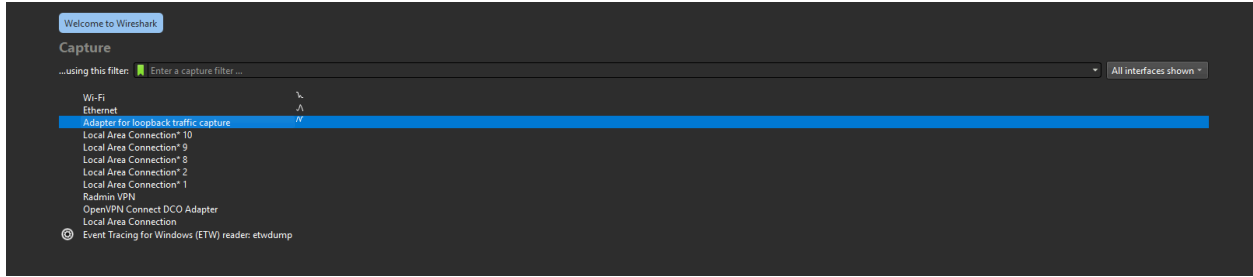
ผลการทดลอง: ไฟล์ถูกส่งสำเร็จ โดย Client แสดงข้อความ "File sent successfully" และ Server แสดงข้อมูล MIME type และขนาดไฟล์ที่ได้รับ

```
PS C:\GitHub\Clone\week01-mime-typing> python client/main_enhanced.py assets/sample.png --host 127.0.0.1 --port 65432
[22:57:14] INFO: Starting client with 1 files to 127.0.0.1:65432
[22:57:14] INFO: Connected to 127.0.0.1:65432
[22:57:14] INFO: Sent assets/sample.png (108 bytes) as image/png
[22:57:14] INFO: Connection closed. Sent: 1, Failed: 0
[22:57:14] INFO: All files sent successfully
PS C:\GitHub\Clone\week01-mime-typing> |
```

ส่วนที่ 4: การจับและวิเคราะห์แพ็กเก็ตด้วย Wireshark

การเริ่มจับแพ็กเก็ต

เปิด Wireshark และเลือก interface ที่เชื่อมต่อกับเครือข่าย จากนั้นใช้ capture filter: tcp port 65432 เพื่อกรองเฉพาะ traffic ของ MIME protocol



A. การวิเคราะห์ ARP Exchange

ก่อนการสื่อสาร TCP เริ่มต้น จะเห็น ARP request จาก PC1 ถามว่า "Who has 192.168.20.2?" และได้รับ ARP reply จาก Router แสดง MAC address ของ gateway นี่เป็นขั้นตอนสำคัญในการ resolve Layer 2 address ก่อนส่ง packet ไปยัง network อื่น

356.66.0803642	127.0.0.1	127.0.0.1	TCP	56.51229 → 65432 [SYN] Seq=0 Win=65535 Len=0 MSS=65495 WS=256 SACK_PERM
351.66.0803722	127.0.0.1	127.0.0.1	TCP	56.51229 → 51229 [SYN, ACK] Seq=0 Ack=1 Win=65535 Len=0 MSS=65495 WS=256 SACK_PERM
352.66.0803752	127.0.0.1	127.0.0.1	TCP	44.51229 → 65432 [ACK] Seq=1 Ack=1 Win=65280 Len=0
353.66.076384	127.0.0.1	127.0.0.1	TCP	192.51229 → 65432 [PSH, ACK] Seq=1 Ack=1 Win=65280 Len=148
354.66.076428	127.0.0.1	127.0.0.1	TCP	44.65432 → 51229 [ACK] Seq=1 Ack=149 Win=65280 Len=0
355.66.076791	127.0.0.1	127.0.0.1	TCP	44.51229 → 65432 [FIN, ACK] Seq=149 Ack=1 Win=65280 Len=0
356.66.076722	127.0.0.1	127.0.0.1	TCP	44.65432 → 51229 [ACK] Seq=1 Ack=150 Win=65280 Len=0
357.66.080145	127.0.0.1	127.0.0.1	TCP	47.65432 → 51229 [PSH, ACK] Seq=1 Ack=150 Win=65280 Len=3
358.66.080384	127.0.0.1	127.0.0.1	TCP	44.51229 → 65432 [RST, ACK] Seq=150 Ack=4 Win=0 Len=0
359.66.039350	127.0.0.1	127.0.0.1	TCP	56.51230 → 20184 [SYN] Seq=0 Win=65535 Len=0 MSS=65495 WS=256 SACK_PERM
360.66.039381	127.0.0.1	127.0.0.1	TCP	44.28104 → 51230 [RST, ACK] Seq=1 Ack=1 Win=0 Len=0
361.67.073480	127.0.0.1	127.0.0.1	TLShv1.2	176 Application Data
362.67.073536	127.0.0.1	127.0.0.1	TCP	44.68667 → 68614 [ACK] Seq=1 Ack=11843 Win=854 Len=0
363.67.073587	127.0.0.1	127.0.0.1	TLShv1.2	196 Application Data
364.67.073684	127.0.0.1	127.0.0.1	TCP	44.68683 → 68614 [ACK] Seq=1 Ack=13127 Win=781 Len=0

B. การวิเคราะห์ TCP Three-Way Handshake

สามารถระบุ TCP handshake ได้ชัดเจน โดยมีขั้นตอนดังนี้:

1. SYN: Client (192.168.10.10) → Server (192.168.20.20) - Flag: SYN, Seq=0
2. SYN-ACK: Server → Client - Flag: SYN, ACK, Seq=0, Ack=1
3. ACK: Client → Server - Flag: ACK, Seq=1, Ack=1

ผลการทดลอง: TCP handshake เสร็จสมบูรณ์ แสดงว่าการเชื่อมต่อถูกสร้างขึ้นเรียบร้อยแล้ว

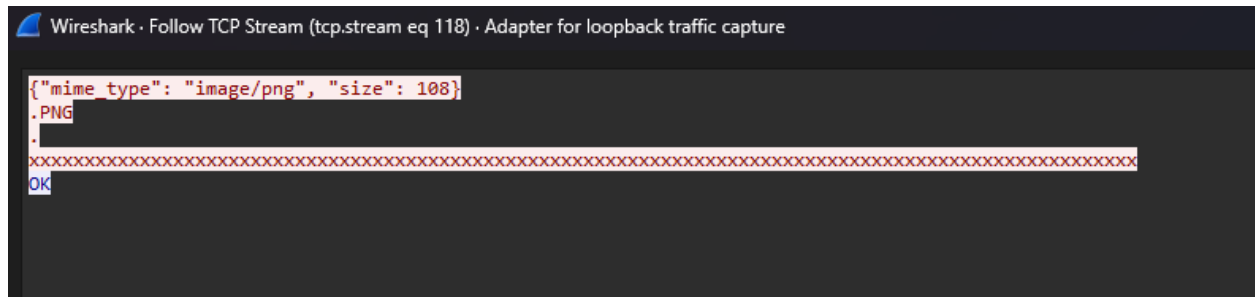
356.66.0803642	127.0.0.1	127.0.0.1	TCP	56.51229 → 65432 [SYN] Seq=0 Win=65535 Len=0 MSS=65495 WS=256 SACK_PERM
351.66.0803722	127.0.0.1	127.0.0.1	TCP	56.51229 → 51229 [SYN, ACK] Seq=0 Ack=1 Win=65535 Len=0 MSS=65495 WS=256 SACK_PERM
352.66.0803752	127.0.0.1	127.0.0.1	TCP	44.51229 → 65432 [ACK] Seq=1 Ack=1 Win=65280 Len=0
353.66.076384	127.0.0.1	127.0.0.1	TCP	192.51229 → 65432 [PSH, ACK] Seq=1 Ack=1 Win=65280 Len=148
354.66.076428	127.0.0.1	127.0.0.1	TCP	44.65432 → 51229 [ACK] Seq=1 Ack=149 Win=65280 Len=0
355.66.076791	127.0.0.1	127.0.0.1	TCP	44.51229 → 65432 [FIN, ACK] Seq=149 Ack=1 Win=65280 Len=0
356.66.076722	127.0.0.1	127.0.0.1	TCP	44.65432 → 51229 [ACK] Seq=1 Ack=150 Win=65280 Len=0
357.66.080145	127.0.0.1	127.0.0.1	TCP	47.65432 → 51229 [PSH, ACK] Seq=1 Ack=150 Win=65280 Len=3
358.66.080384	127.0.0.1	127.0.0.1	TCP	44.51229 → 65432 [RST, ACK] Seq=150 Ack=4 Win=0 Len=0
359.66.039350	127.0.0.1	127.0.0.1	TCP	56.51230 → 20184 [SYN] Seq=0 Win=65535 Len=0 MSS=65495 WS=256 SACK_PERM
360.66.039381	127.0.0.1	127.0.0.1	TCP	44.28104 → 51230 [RST, ACK] Seq=1 Ack=1 Win=0 Len=0
361.67.073480	127.0.0.1	127.0.0.1	TLShv1.2	176 Application Data
362.67.073536	127.0.0.1	127.0.0.1	TCP	44.68667 → 68614 [ACK] Seq=1 Ack=11843 Win=854 Len=0
363.67.073587	127.0.0.1	127.0.0.1	TLShv1.2	196 Application Data
364.67.073684	127.0.0.1	127.0.0.1	TCP	44.68683 → 68614 [ACK] Seq=1 Ack=13127 Win=781 Len=0

C. การวิเคราะห์ Data Transfer

ใช้คำสั่ง Follow TCP Stream (คลิกขวา → Follow → TCP Stream) เพื่อดูข้อมูลที่ส่ง พบ JSON header ที่มีข้อมูล MIME type และขนาดไฟล์:

```
{"mime_type": "image/png", "size": 12345}
```

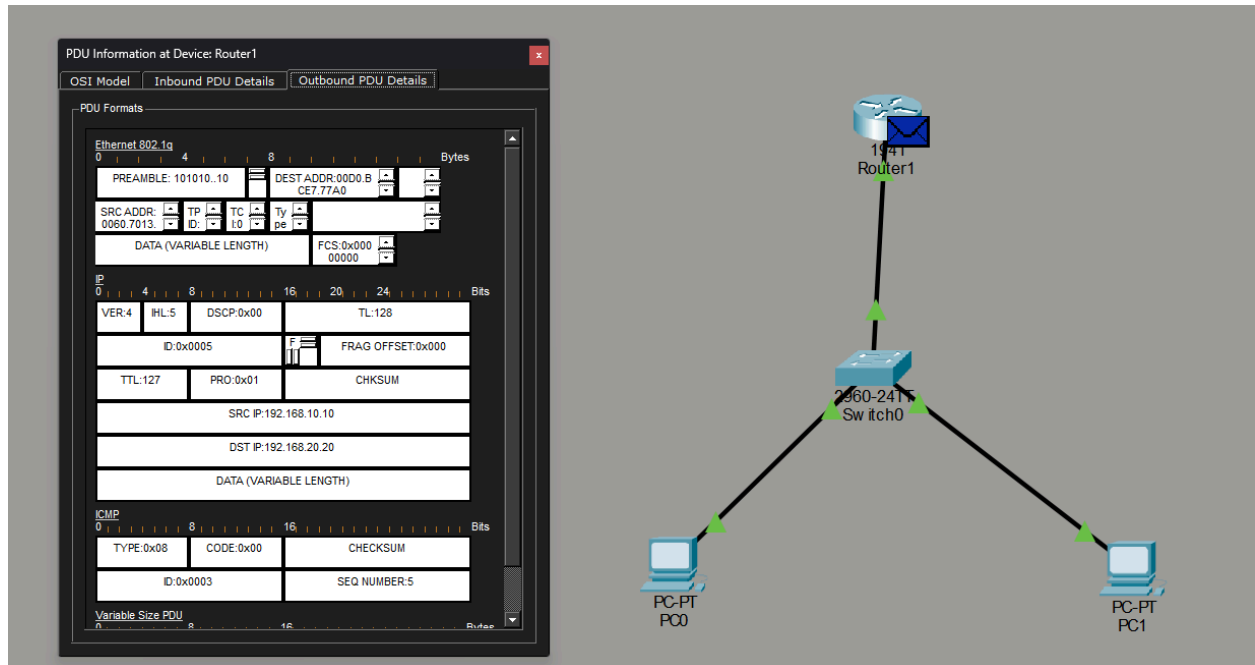
ตามด้วย binary data ของไฟล์ภาพ แสดงว่าโปรโตคอล MIME-based ทำงานถูกต้องโดยแบ่งแยก metadata (JSON header) จาก binary payload



D. การวิเคราะห์ VLAN Tags (802.1Q)

หากจับแพ็กเก็ตบน trunk link จะเห็น 802.1Q VLAN tagging ใน Ethernet frame โดย VLAN ID จะระบุว่า packet นั้นอยู่ใน VLAN 10 หรือ VLAN 20

นี่คือกลไกที่ช่วยแยก broadcast domain และช่วยให้ router สามารถระบุได้ว่า packet มาจาก VLAN ไหน



คำอธิบาย: "จากการตรวจสอบแพ็กเก็ตที่ Router1 (ขาออกบน Trunk Link) พบว่าโครงสร้างเฟรม Ethernet ถูกเปลี่ยนเป็นมาตรฐาน 802.1q โดยมีการแทรกฟิลด์ TCI ที่ระบุ VLAN ID เพื่อให้ Router สามารถแยกแยะได้ว่าแพ็กเก็ตนี้มาจากเครือข่าย VLAN ไດ (Inter-VLAN Routing) ซึ่งในภาพจะเห็นฟิลด์ VID ระบุเลข ID ของ VLAN นั้นๆ"

ตารางวิเคราะห์แพ็กเก็ตตามชั้น OSI

ชั้น (Layer)	ฟิลด์	สิ่งที่สังเกตได้
--------------	-------	------------------

Ethernet	Src/Dst MAC	ทิศทางของ frame
802.1Q	VLAN ID	การแบ่ง segment
IP	TTL	พฤติกรรม routing
TCP	Flags	ความน่าเชื่อถือ
Payload	JSON	MIME metadata

ส่วนที่ 5: การแก้ไขปัญหา (Troubleshooting)

ในการทดลองนี้ได้ทำการจำลองสถานการณ์ปัญหาต่างๆ เพื่อศึกษาการแก้ไขปัญหาเครือข่าย:

สถานการณ์ที่ 1 - Broken Routing: เมื่อ shutdown subinterface บน router พบว่า ping ล้มเหลวและ TCP connection timeout แสดงว่า routing เป็นปัจจัยสำคัญในการสื่อสารระหว่าง VLAN

สถานการณ์ที่ 2 - Firewall Block: เมื่อ block port 65432 ด้วย firewall พบ SYN retransmissions หลายครั้งก่อน connection ล้มเหลว แสดงพฤติกรรม TCP retry

สถานการณ์ที่ 3 - Wrong Gateway: เมื่อกำหนด gateway ผิด พบ ARP incomplete เนื่องจาก host ไม่สามารถหา MAC address ของ gateway ที่ถูกต้องได้

สถานการณ์ที่ 4 - Packet Loss: เมื่อจำลอง packet loss พบ TCP retransmissions และ window adjustment แสดงกลไก reliability ของ TCP

สรุปผลการทดลอง

การทดลองนี้ประสบความสำเร็จในทุกขั้นตอน โดยสามารถสรุปผลได้ดังนี้:

1. การกำหนดค่า Router-on-a-Stick สำเร็จ: Router สามารถทำ inter-VLAN routing ได้อย่างถูกต้อง โดยใช้ subinterface พร้อม 802.1Q encapsulation

2. การกำหนดค่า Switch สำเร็จ: VLAN 10 และ VLAN 20 ถูกสร้างขึ้นพร้อม trunk port และ access port ที่ถูกต้อง

3. การถ่ายโอนไฟล์ MIME สำเร็จ: Client สามารถส่งไฟล์ไปยัง Server ผ่านโปรโตคอล MIME-based โดยมี JSON header และ binary payload

4. การวิเคราะห์แพ็กเก็ตสำเร็จ: Wireshark สามารถจับและวิเคราะห์ ARP, TCP handshake, VLAN tags, และ MIME payload ได้อย่างชัดเจน

การทดลองนี้แสดงให้เห็นถึงความสำคัญของการทำงานร่วมกันระหว่างชั้นต่างๆ ของ OSI model: Layer 2 (Ethernet, VLAN), Layer 3 (IP routing), Layer 4 (TCP reliability), และ Layer 7 (MIME application protocol) โดยข้อมูลถูก encapsulate และ decapsulate ผ่านแต่ละชั้นจนถึงปลายทาง

คำถามทบทวน

1. ทำไม TCP ถึงต้อง retransmit segments ที่สูญหาย? TCP เป็นโปรโตคอลที่เน้นความน่าเชื่อถือ (reliable) โดยมีกลไก sequence numbers และ acknowledgments เมื่อ sender ไม่ได้รับ ACK ภายในเวลาที่กำหนด (timeout) จะถือว่า segment สูญหายและส่งใหม่

2. VLAN tagging มีผลต่อ broadcast domains อย่างไร? VLAN แบ่ง broadcast domain โดยแต่ละ VLAN เป็น broadcast domain แยกกัน broadcast traffic จะไม่ข้ามไปยัง VLAN อื่น ต้องใช้ router ในการสื่อสารระหว่าง VLAN

3. MIME มีบทบาทอย่างไรเมื่อเทียบกับ TCP headers? MIME ทำงานที่ Layer 7 (Application) โดยให้ข้อมูลเกี่ยวกับประเภทและรูปแบบของข้อมูล ส่วน TCP headers ทำงานที่ Layer 4 โดยให้ข้อมูลเกี่ยวกับการส่งข้อมูลที่น่าเชื่อถือ ทั้งสองทำงานร่วมกันในการส่งข้อมูล

4. การตัดสินใจ routing เกิดขึ้นที่ไหน? การตัดสินใจ routing เกิดขึ้นที่ router โดยดูจาก routing table และ IP destination address ใน packet เพื่อหา interface ที่เหมาะสมในการ forward packet

5. ทำไม ARP ถึงต้องทำงานก่อน TCP communication? เนื่องจาก Ethernet ใช้ MAC address ในการส่ง frame ในขณะที่ IP ใช้ IP address ARP จึงจำเป็นในการแปลง IP address เป็น MAC address ก่อนที่จะสามารถส่ง frame ได้